

Toolformer: Language Models Can Teach Themselves to Use Tools

Step 1: 采样 API 调用 (Sampling)

做什么：让模型自己在文本中标注"哪里可能需要调用工具"

怎么做：

1. 给模型一个 few-shot prompt (几个示例)
2. 对文本中每个位置 i ，计算模型生成 $\langle \text{API} \rangle$ 的概率： $p_i = p_M(\text{API} \mid P(x), x_{1:i-1})$
3. 概率超过阈值 τ_s 的位置保留，采样生成具体的 API 调用

原来调用什么API，这点本身也是可以训练出来的

Step 3: 过滤无用调用 (Filtering) ——精华所在！

核心思想：只保留"真正有用"的 API 调用

怎么判断有用？用损失函数的变化：

$$L_i^+ = L_i(e(c_i, r_i)) \quad (\text{有API调用+结果时的损失})$$

$$L_i^- = \min(L_i(e), L_i(e(c_i, e))) \quad (\text{无调用或无结果时的损失})$$

过滤条件：

$$L_i^- - L_i^+ \geq \tau_f$$

通俗理解：

- 如果给了工具结果后，模型预测后续文本**更准确**（损失降低）→ 保留
- 如果有没有结果都差不多 → 丢弃

如何判断调用的东西是有用的，这点也需要训练，过滤，这一点是训练的过程，在实际的调用中应该没有过滤

⚠ 局限性（需要了解）

1. **单次调用限制**：每个输入只能调用一次 API
2. **无法链式推理**：不能"先查日期，再用日期查事件"
3. **搜索引擎较弱**：简单的搜索难以处理复杂查询
4. **需要一定规模**：<775M 参数的模型学不会用工具

现在肯定已经解决掉这些问题了，时代发展的真的快

✅ 总结：三句话记住 Toolformer

1. **核心贡献**：首次让 LLM 以自监督方式学会使用多种工具
2. **关键创新**：用"损失减少量"自动筛选有用的工具调用
3. **重要意义**：小模型 + 工具 > 大模型（以小博大的典范）

其实这篇论文也就是将这个调用API本身加入到这个训练之中去

例子 2：浏览器里找信息（网页操作）

人类 UI

看到网页 → 滚动 → 找按钮 → 点开 → 复制一段文字

ACI（两种常见形式）

形式 A：文本化网页（推荐）

- 观察：网页被转换成“可读文本 + 元素结构（标题/链接/表单）”
- 动作：`click(link_id)`、`type(input_id, "xxx")`、`scroll()`
优点：**不怕网页布局变化**，不会“点歪”。

形式 B：截图 + 坐标点击（更像真的点屏幕）

- 观察：截图
- 动作：点击某个坐标 `(x,y)`
缺点：网页改一点点布局，坐标就废了；不稳定。

所以 ACI 的核心价值之一：把“视觉 UI”转换成“结构化可操作对象”。